

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



C# PROGRAMMING LANGUAGE

Bài tập lớn:

Media Streaming Web Application

Môn học: C# and .NET Development

Học kỳ: Học kỳ 3 2026

TP. HỒ CHÍ MINH, NĂM 2026

TuneVault

Media Streaming Web Application

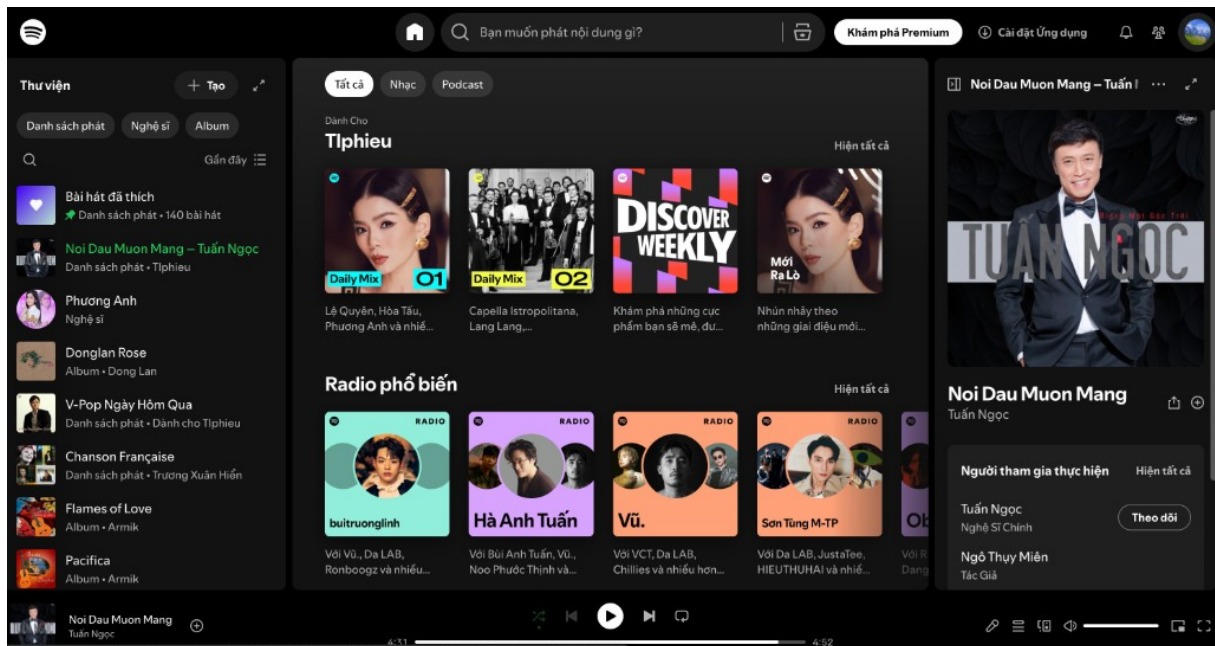
Assignment Overview

- **Tổng điểm:** 10 điểm (Frontend 2 điểm + Backend 8 điểm)
- **Loại hình:** Bài tập lớn thực hiện theo nhóm (tối đa 6 sinh viên)
- **Công nghệ:** React + TypeScript (Frontend) + ASP.NET Core (Backend)
- **Cơ sở dữ liệu:** Entity Framework Core *hoặc* Dapper (chọn một)
- **Thời gian khuyến nghị:** 7 tuần

Mô tả ứng dụng — TuneVault

TuneVault là nền tảng phát nhạc và video trực tuyến. Giao diện người dùng **tham khảo Spotify**:

- Thanh điều hướng bên trái
- Vùng nội dung trung tâm: (playlist, album, gợi ý)
- Panel phải: (chi tiết bài hát/nghệ sĩ)
- Player bar cố định phía dưới
- Sinh viên triển khai đầy đủ chức năng audio và video, đồng thời có tính năng chia sẻ media giữa người dùng và **thông báo** real-time.



Hình 1: Giao diện tham khảo (Spotify) — layout sinh viên cần mô phỏng cho TuneVault

Contents

1	Giới thiệu	2
1.1	Mục tiêu học tập	2
1.2	Mô tả ứng dụng	2
2	Yêu cầu kỹ thuật	2
2.1	Công nghệ bắt buộc	2
2.1.1	Frontend (2 điểm)	2
2.1.2	Backend (8 điểm)	3
2.1.3	Bonus — CI/CD & Cloud	3
2.2	Kiến trúc Clean Architecture	3
3	Application Pipeline	4
4	Danh sách 10 chức năng bắt buộc	5
5	Thiết kế cơ sở dữ liệu	5
5.1	Entity gợi ý (tối thiểu 10 bảng)	5
6	Yêu cầu giao diện Frontend (2 điểm)	6
6.1	F1 — Giao diện kiểu Spotify & Media Players <i>(1,0 điểm)</i>	6
6.2	F2 — Tích hợp API & Trạng thái ứng dụng <i>(1,0 điểm)</i>	6
7	Yêu cầu Backend (8 điểm)	6
7.1	B1 — Clean Architecture & Cấu trúc solution <i>(1,0 điểm)</i>	6
7.2	B2 — Entity Framework Core HOẶC Dapper <i>(1,0 điểm)</i>	7
7.3	B3 — RESTful API, DTO, Swagger/Postman <i>(1,0 điểm)</i>	7
7.4	B4 — JWT Authentication & Authorization <i>(0,5 điểm)</i>	7
7.5	B5 — Upload & Streaming Media (Audio + Video) <i>(1,0 điểm)</i>	7
7.6	B6 — Chia sẻ Media giữa người dùng <i>(0,5 điểm)</i>	7
7.7	B7 — Thông báo (SignalR + lưu CSDL) <i>(1,0 điểm)</i>	8
7.8	B8 — Application Pipeline cho 10 chức năng	8
8	Tích hợp AI <i>(1,0 điểm)</i>	8
8.1	Các tính năng AI gợi ý	8
8.2	Hướng dẫn kỹ thuật tích hợp	8
8.3	Điểm thưởng cho tính năng AI	10
9	Bonus: CI/CD & Cloud	10
10	Bảng phân bổ điểm tổng hợp	11
11	Hướng dẫn nộp bài	11
11.1	Tài liệu bắt buộc	11
11.2	Dealine nộp bài	11
12	Xử lý gian lận	12

1 Giới thiệu

1.1 Mục tiêu học tập

Sau khi hoàn thành bài tập lớn, sinh viên có khả năng:

- Thiết kế và triển khai **Clean Architecture** trên ASP.NET Core Web API.
- Làm việc với cơ sở dữ liệu quan hệ bằng **Entity Framework Core** hoặc **Dapper** (chọn **một**, dùng nhất quán toàn dự án).
- Xây dựng RESTful API, JWT Authentication, upload/stream media (audio & video).
- Tích hợp frontend React (TypeScript) giao diện kiểu Spotify.
- Triển khai **Application Pipeline** cho từng chức năng nghiệp vụ.
- Kiểm thử API bằng **Swagger** hoặc **Postman**.
- (Tích hợp **AI generative** (Anthropic Claude API) vào use case thực tế.
- (**Bonus**) Triển khai CI/CD và đưa ứng dụng lên cloud.

1.2 Mô tả ứng dụng

TuneVault cung cấp các chức năng chính sau cho người dùng (giao diện tham khảo: **Hình 1**, trang 2):

- Đăng ký, đăng nhập, quản lý hồ sơ cá nhân.
- Tải lên và phát file **audio** (mp3, wav, ...) và **video** (mp4, webm, ...).
- Tạo playlist, tìm kiếm, theo dõi người dùng/nghệ sĩ.
- **Chia sẻ** bài hát/playlist/video cho người dùng khác (bắt buộc).
- Nhận **thông báo** khi có lượt chia sẻ, lời mời, hoặc tương tác liên quan.

2 Yêu cầu kỹ thuật

2.1 Công nghệ bắt buộc

2.1.1 Frontend (2 điểm)

- React 18+ với **TypeScript**.
- React Router v6+, Axios hoặc Fetch API.
- CSS: Tailwind CSS hoặc CSS Modules (giao diện tối, layout kiểu Spotify).
- Trình phát: HTML5 <audio> / <video> hoặc thư viện (react-h5-audio-player, video.js, ...).
- Client SignalR cho thông báo real-time .
- Build tool: Vite.

2.1.2 Backend (8 điểm)

- ASP.NET Core 8+ Web API.
- **Clean Architecture**: Domain, Application, Infrastructure, API (Presentation).
- ORM: **Entity Framework Core 8+ HOẶC Dapper** (bắt buộc chọn một).
- CSDL: SQL Server (LocalDB hoặc instance đầy đủ).
- Xác thực: JWT + ASP.NET Core Identity (khuyến khích).
- Tài liệu API: Swagger/OpenAPI (/swagger).
- Real-time: SignalR (Hub thông báo).
- Lưu file media: thư mục local hoặc Azure Blob Storage (nếu làm bonus cloud).

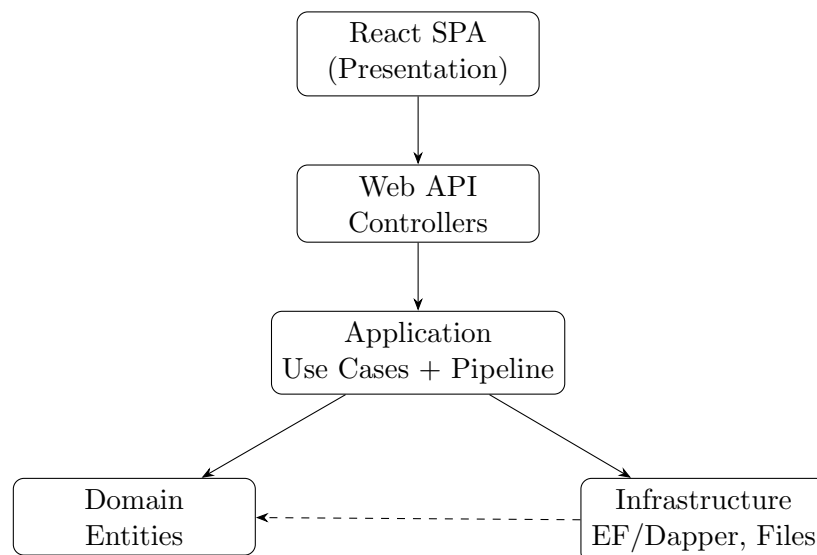
2.1.3 Bonus — CI/CD & Cloud

- Triển khai lên Azure App Service / AWS / Docker + VPS.
- Pipeline GitHub Actions hoặc Azure DevOps (build, test, deploy).
- Điểm cộng tối đa: **+1,0 điểm** (theo rubric mục 9).

2.2 Kiến trúc Clean Architecture

Sinh viên tổ chức solution theo Clean Architecture với **4 project độc lập** (xem chi tiết yêu cầu chấm điểm tại mục B1 ??).

```
TuneVault.sln
|-- TuneVault.Domain          (Entities, Enums, Domain Events,
    Interfaces)
|-- TuneVault.Application     (Use Cases, DTOs, Validators,
    Pipeline Behaviors)
|-- TuneVault.Infrastructure  (EF/Dapper, Repositories, File
    Storage, SignalR)
|-- TuneVault.API             (Controllers, Middleware, DI,
    Swagger)
```



Quy tắc phụ thuộc: Domain không phụ thuộc layer khác; Application chỉ phụ thuộc Domain; Infrastructure implement interface từ Application/Domain; API chỉ gọi Application, không gọi trực tiếp DbContext trong Controller.

3 Application Pipeline

Mỗi trong **10 chức năng** (mục 4) phải được triển khai qua **một pipeline xử lý request** thống nhất tại tầng Application. Sinh viên có thể dùng:

- **MediatR** với `IPipelineBehavior<TRequest, TResponse>`, hoặc
- Pipeline tự xây: chuỗi bước `Validate` → `Authorize` → `Execute` → `Map Response`.

Pipeline tối thiểu cho mỗi use case:

1. **Validation** — `FluentValidation` hoặc `DataAnnotations` trên Request DTO.
2. **Authorization** — kiểm tra JWT, quyền sở hữu tài nguyên (ví dụ: chỉ chủ playlist mới sửa).
3. **Handler / Use Case** — logic nghiệp vụ, gọi repository qua interface.
4. **Persistence** — EF Core hoặc Dapper trong Infrastructure.
5. **Side effects** — ghi notification, publish domain event (nếu có).
6. **Response mapping** — trả DTO chuẩn hóa (không trả entity thô).

Ví dụ — *Chia sẻ bài hát*:

```

ShareTrackCommand
-> ShareTrackValidator           % validate input DTO
-> ShareTrackAuthorizationBehavior % check sender != receiver
-> ShareTrackHandler             % create Share + Notification
    records
-> ShareTrackResponseDto         % return standardized response
  
```

Sinh viên nộp kèm **sơ đồ pipeline** (1 trang) cho ít nhất **3 chức năng** tiêu biểu: Auth, Share Media, Notifications.

4 Danh sách 10 chức năng bắt buộc

Mỗi chức năng gồm: API backend (Swagger/Postman), pipeline Application, và màn hình React tương ứng (có thể gộp UI nếu hợp lý).

#	Chức năng	Mô tả	Pipeline / API gợi ý
1	Xác thực	Đăng ký, đăng nhập, refresh token (tùy chọn), đăng xuất.	Register/LoginCommand + JWT issuance.
2	Hồ sơ người dùng	Xem/sửa profile, avatar, bio.	UpdateProfileCommand, GET profile.
3	Thư viện Media	Upload metadata + file audio/video; phân loại bài hát, album.	UploadMediaCommand, streaming endpoint.
4	Audio Player	Phát nhạc, pause, seek; hiển thị đang phát; queue đơn giản.	GET /media/{id}/stream; frontend player bar; RecordPlayHistory khi bắt đầu phát.
5	Video Player	Phát video full-page hoặc modal; poster thumbnail.	GET video stream / range requests (khuyến khích).
6	Playlist	CRUD playlist; thêm/xóa track; playlist công khai/riêng tư.	CreatePlaylist, AddTrackToPlaylist.
7	Tìm kiếm & Khám phá	Tìm theo tên bài, nghệ sĩ, playlist; gợi ý trending (đơn giản).	SearchMediaQuery với phân trang.
8	Chia sẻ Media (*)	Gửi bài hát/playlist/video cho user khác; xem “Đã chia sẻ với tôi”.	ShareMediaCommand; bảng MediaShare.
9	Thông báo (*)	Thông báo khi được chia sẻ, được follow, playlist được share.	SignalR Hub + Notification entity; mark as read.
10	Tương tác & Lịch sử	Like/favorite track; lịch sử nghe gần đây (10 bài mới nhất).	ToggleFavoriteCommand, RecordPlayHistoryCommand.

Table 1: 10 chức năng bắt buộc — mỗi chức năng phải có pipeline đầy đủ

(*) Chức năng 8 và 9 là **bắt buộc** và sẽ được chấm kỹ hơn trong rubric.

5 Thiết kế cơ sở dữ liệu

5.1 Entity gợi ý (tối thiểu 10 bảng)

- `AspNetUsers` / `UserProfile`
- `MediaItem` (audio/video, duration, file path, owner)

- Album, Artist (có thể gộp vào MediaItem nếu đơn giản)
- Playlist, PlaylistTrack
- MediaShare (sender, receiver, media/playlist id, shared at)
- Notification (user id, type, payload JSON, is read)
- Favorite
- PlayHistory
- Follow (user follows user hoặc artist)

Sinh viên nộp **sơ đồ ERD** (draw.io, dbdiagram.io, hoặc SSMS diagram). Nếu dùng EF: ít nhất **2 migration**; nếu dùng Dapper: script `.sql` tạo bảng + seed data.

6 Yêu cầu giao diện Frontend (2 điểm)

6.1 F1 — Giao diện kiểu Spotify & Media Players (1,0 điểm)

- Layout: sidebar trái (Home, Search, Library), vùng nội dung, **player bar** cố định phía dưới.
- Theme tối (dark) hoặc dark/light toggle.
- Responsive: desktop + tablet (mobile khuyến khích).
- Trang riêng hoặc view cho **video player** (khác audio bar).
- Tối thiểu **8 màn hình/route** có ý nghĩa (Login, Home, Search, Library, Playlist Detail, Share Inbox, Notifications, Profile).

6.2 F2 — Tích hợp API & Trạng thái ứng dụng (1,0 điểm)

- Layer service API (Axios) + TypeScript interfaces khớp DTO backend.
- Lưu JWT (httpOnly cookie khuyến khích, hoặc localStorage có giải thích bảo mật).
- Protected routes; redirect khi hết phiên.
- Kết nối SignalR cho thông báo (badge số chưa đọc).
- Loading/error state cho upload và phát media.

7 Yêu cầu Backend (8 điểm)

7.1 B1 — Clean Architecture & Cấu trúc solution (1,0 điểm)

Cấu trúc solution tham khảo mục 2.2. Tiêu chí chấm điểm:

- Đủ 4 project/layer; dependency rule đúng (Domain $\leftarrow$ Application $\leftarrow$ Infrastructure, API).

- DI đăng ký trong `Program.cs` hoặc `DependencyInjection.cs`; không hardcode dependency.
- Không có logic nghiệp vụ trong Controller (chỉ gọi MediatR command/query handler).
- Mỗi layer chỉ reference layer bên trong (không reference ngược).

7.2 B2 — Entity Framework Core HOẶC Dapper (1,0 điểm)

- **EF Core:** DbContext, Fluent API, migrations, repository (tùy chọn).
- **Dapper:** interface repository, parameterized queries, transaction khi cần.
- Seed data mẫu: ít nhất 2 user, **10 media items** (mix audio và video), 2 playlist.

7.3 B3 — RESTful API, DTO, Swagger/Postman (1,0 điểm)

- Tối thiểu **20 endpoints** có ý nghĩa (CRUD + stream + share + notify); không tính các endpoint trùng lặp hoặc chỉ đổi HTTP method.
- DTO request/response; không lộ entity.
- Swagger mô tả đủ; **hoặc** file Postman Collection export kèm screenshot chạy test.
- Response format thống nhất (ví dụ: { success, data, errors }).
- CORS cấu hình cho origin React.

7.4 B4 — JWT Authentication & Authorization (0,5 điểm)

- Register/Login trả JWT.
- [Authorize] trên endpoint nhạy cảm.
- Chỉ chủ sở hữu mới sửa/xóa media/playlist của mình.

7.5 B5 — Upload & Streaming Media (Audio + Video) (1,0 điểm)

- Upload multipart; giới hạn kích thước (cấu hình).
- Lưu metadata vào SQL; file lưu disk/blob.
- Endpoint phát stream (hỗ trợ Range header cho video).
- Validate định dạng file (whitelist extension/MIME).

7.6 B6 — Chia sẻ Media giữa người dùng (0,5 điểm)

- API: chia sẻ theo receiverUserId + mediaId hoặc playlistId.
- Danh sách “shared with me” / “shared by me”.
- Pipeline đầy đủ; kiểm tra receiver tồn tại; không chia sẻ trùng vô hạn (idempotent tùy chọn).

7.7 B7 — Thông báo (SignalR + lưu CSDL)

(1,0 điểm)

- Khi share thành công → tạo notification + push qua SignalR.
- API: list notifications, mark as read.

7.8 B8 — Application Pipeline cho 10 chức năng

- Mỗi chức năng mục 4 có ít nhất 1 command/query + pipeline behaviors.
- Tài liệu gắn (README) mô tả pipeline từng feature.

8 Tích hợp AI

(1,0 điểm)

Sinh viên có thể bổ sung **một hoặc nhiều tính năng AI** vào TuneVault bằng cách gọi **Anthropic API** trực tiếp từ tầng Application/Infrastructure của ASP.NET Core. Đây là tính năng **không bắt buộc** nhưng được khuyến khích mạnh vì thể hiện khả năng tích hợp dịch vụ bên ngoài trong kiến trúc Clean Architecture.

8.1 Các tính năng AI gợi ý

1. Gợi ý bài hát thông minh (AI Recommendation)

Dựa vào lịch sử nghe (PlayHistory) và danh sách yêu thích (Favorite) của người dùng, gửi prompt tới Claude để nhận danh sách *title/artist* gợi ý, sau đó tra cứu trong CSDL để trả về *MediaItem* thực sự.

Endpoint: GET /ai/recommendations

2. Tóm tắt & mô tả bài hát (AI Description)

Khi upload media, sinh viên có thể gửi tên bài, nghệ sĩ, thể loại tới Claude để tự động sinh mô tả ngắn (50–100 từ) lưu vào trường *MediaItem.Description*.

Endpoint: POST /ai/generate-description

3. Chatbot hỗ trợ người dùng (TuneBot)

Một chatbot đơn giản trả lời câu hỏi về cách dùng app, tìm kiếm bài hát, hoặc giải thích tính năng chia sẻ. Frontend hiển thị dưới dạng floating chat bubble.

Endpoint: POST /ai/chat

4. Phân loại tự động (Auto-tagging)

Gửi tên bài + nghệ sĩ tới Claude để nhận danh sách tag thể loại (*Pop, Lo-fi, Jazz, ...*), lưu vào bảng *MediaTag*.

Endpoint: POST /ai/auto-tag/{mediaId}

8.2 Hướng dẫn kỹ thuật tích hợp

Kiến trúc tích hợp đúng Clean Architecture:

```
Application/
  AI/
    IAnthropicService.cs           % interface (Domain-agnostic)
    GetRecommendationsQuery.cs    % MediatR query
    GetRecommendationsHandler.cs
```

```

Infrastructure/
  AI/
    AnthropicService.cs          % gọi Anthropic REST API
    AnthropicOptions.cs          % bind tu appsettings.json

API/
  Controllers/
    AiController.cs              % chỉ gọi MediatR, không gọi SDK
    trực tiếp

```

Ví dụ gọi Anthropic API từ C#:

```

// Infrastructure/AI/AnthropicService.cs
public class AnthropicService : IAnthropicService
{
    private readonly HttpClient _http;
    private readonly string _apiKey;

    public async Task<string> CompleteAsync(string prompt,
        CancellationToken ct = default)
    {
        var body = new
        {
            model = "claude-sonnet-4-20250514",
            max_tokens = 512,
            messages = new[]
            {
                new { role = "user", content = prompt }
            }
        };

        using var req = new HttpRequestMessage(
            HttpMethod.Post,
            "https://api.anthropic.com/v1/messages");
        req.Headers.Add("x-api-key", _apiKey);
        req.Headers.Add("anthropic-version", "2023-06-01");
        req.Content = JsonContent.Create(body);

        var res = await _http.SendAsync(req, ct);
        var json = await res.Content.ReadFromJsonAsync
            <AnthropicResponse>(cancellationToken: ct);
        return json?.Content?[0]?.Text ?? string.Empty;
    }
}

```

Cấu hình API key an toàn (không hardcode, không commit lên Git):

```

// appsettings.Development.json (thêm vào .gitignore)
{
    "Anthropic": {
        "ApiKey": "sk-ant-..."
    }
}

```

```

}

// Program.cs
builder.Services.Configure<AnthropicOptions>(
    builder.Configuration.GetSection("Anthropic"));
builder.Services.AddHttpClient<IAntropicService,
    AnthropicService>();

```

8.3 Điểm thưởng cho tính năng AI

Tính năng AI hoàn chỉnh	Điểm cộng
1 tính năng AI hoạt động, có pipeline đúng kiến trúc	+0,3
2+ tính năng AI; interface đúng DI; key không hardcoded	+0,5
Thêm streaming response (SSE) cho chatbot	+0,2
Tổng AI bonus tối đa	+0,5

Table 2: Điểm thưởng tính năng AI

Sinh viên **bắt buộc** sử dụng một trong hai:

- **Swagger UI** — demo đầy đủ luồng Auth → Upload → Share → Notify.
- **Postman** — collection có biến môi trường (`baseUrl`, `token`), ít nhất 15 request có test script/assertion.

Nộp kèm: screenshot hoặc video ngắn (≤ 3 phút) demo API, hoặc file `.http` / Postman export.

9 Bonus: CI/CD & Cloud

Lưu ý: Điểm bonus CI/CD và điểm bonus AI (mục 8)

Hạng mục	Điểm cộng tối đa
Pipeline CI (build + test) trên GitHub Actions / Azure DevOps	+0,25
Deploy backend lên cloud + SQL managed	+0,25
Deploy frontend (Static Web Apps / Vercel / Netlify)	+0,25
Tài liệu hướng dẫn deploy đầy đủ	+0,25
Tính năng AI (xem mục 8 để biết chi tiết)	+0,5 – +1,0

Table 3: Điểm thưởng

Hạng mục	Điểm	Tỷ lệ
Frontend (F1 + F2)	2,0	20%
Backend (B1 – B8)	8,0	80%
Tổng	10,0	100%

Table 4: Phân bổ điểm chính

Mã	Tiêu chí	Điểm
F1	UI Spotify-like, audio/video players	1,0
F2	API integration, auth, SignalR client	1,0
B1	Clean Architecture	1,0
B2	EF Core hoặc Dapper	1,5
B3	REST, DTO, Swagger/Postman	1,0
B4	JWT Auth	1,0
B5	Media upload/stream	1,0
B6	Share media	1,0
B7	Notifications	0,5
B8	Pipeline 10 features	0,0

Table 5: Chi tiết rubric chấm điểm

10 Bảng phân bổ điểm tổng hợp

11 Hướng dẫn nộp bài

11.1 Tài liệu bắt buộc

- File nén `MSSV_TenSV_TuneVault.zip` chứa:
 - Solution .NET (.sln) + project React.
 - `README.md`: hướng dẫn chạy local, connection string, tài khoản seed.
 - ERD + sơ đồ pipeline (PDF hoặc PNG).
 - Swagger export hoặc Postman Collection.
- Báo cáo ngắn (5–10 trang PDF): kiến trúc, lựa chọn EF/Dapper, mô tả 10 chức năng, khó khăn & hướng xử lý.
- Video demo (khuyến khích, 5–8 phút): UI + API + share + notification.

11.2 Dealine nộp bài

- Deadline: **23:59, ngày 20/06/2026 (thứ Bảy)**.
- Nộp vào folder Assignment, trong folder này mỗi nhóm nên tạo một folder riêng đặt tên nhóm của mình; tên file đúng định dạng. Tất cả source code phải được push lên GitHub (private repo được, nhưng phải có commit history rõ ràng giữa các thành viên).

- **Không chấp nhận nộp trễ.**
- Những nhóm nộp bài trễ nếu mà có lý do chính đáng (bệnh tật, sự cố kỹ thuật) sẽ được xem xét riêng và không được hưởng bất cứ quyền lợi gì, dù source có phát triển thêm hay không.

12 Xử lý gian lận

- Nếu phát hiện sao chép code giữa sinh viên, cả hai sẽ bị điểm 0 cho phần liên quan.
- Sinh viên phải hiểu và giải thích được code.
- Trích dẫn thư viện, template, snippet nguồn mở trong README.

References

- [1] ASP.NET Core. Available: <https://learn.microsoft.com/aspnet/core>
- [2] Entity Framework Core. Available: <https://learn.microsoft.com/ef/core>
- [3] Dapper. Available: <https://github.com/DapperLib/Dapper>
- [4] MediatR (pipeline pattern). Available: <https://github.com/jbogard/MediatR>
- [5] FluentValidation. Available: <https://docs.fluentvalidation.net>
- [6] Clean Architecture (Jason Taylor template). Available: <https://github.com/jasontaylordev/CleanArchitecture>
- [7] SignalR. Available: <https://learn.microsoft.com/aspnet/core/signalr>
- [8] Anthropic API (Claude) — tích hợp AI. Available: <https://docs.anthropic.com/en/api/getting-started>
- [9] React. Available: <https://react.dev>
- [10] Vite. Available: <https://vitejs.dev>
- [11] Rubric B1: "Clean Architecture & Cấu trúc solution" — Chi tiết tiêu chí chấm điểm cho mục B1 (xác định 4 project, dependency rule, DI, và việc tách biệt logic nghiệp vụ).